SIMATS ENGINEERING

CO3- ASSESSMENT TOOL 2

SIMULATION TASK

COURSE NAME: DIGITAL SIGNAL PROCESSING   COURSE CODE: ECA09

COURSE OUTCOME COVERED

CO 3: Evaluate finite word-length effects and multirate signal processing techniques, including decimation, interpolation, and polyphase decomposition. **Bloom Level mapped-BL3,BL4,BL5**

Topics Covered:

Quantization- Truncation and Rounding errors - Quantization noise - Limit cycle oscillations – Signal scaling.Parametric and Non parametric Spectral estimation. Decimation-Interpolation-Sampling rate conversion-Implementation of sampling rate conversion-Polyohase Decomposition -5G Decimation/Interpolation: For mmWave beamforming.

Assessment Tool number	**: CO3-AT2**
Assessment Tool Weightage	**: 35%**
Assessment Tool Name	**: Simulation Task**
Duration of this assessment	**: 60 Minutes**
Marks	**: 25 Marks**
Type of Assessment Conduction	**: Self Learning (Home Task)**

- Each student will be assigned one simulation-based problem related to the course topics.
- Solve the assigned simulation problem independently.
- Develop and run the simulation using MATLAB/Simulink.
- Analyze and present the results with graphs and screenshots.
- Upload the simulation code and output to GitHub.
- Submit the GitHub repository link in Google Classroom (GCR).

QUESTION:

23. Combined Decimation and Interpolation

Simulate a multirate system using decimation and interpolation and analyze the resulting sampling rate conversion. (25) (BL4) (WK4)

Given Data:

- Decimation Factor = 3
- Interpolation Factor = 2
- Input Sampling Rate = 18 kHz

Tasks:

1. Design an anti-alias filter.
2. Perform decimation.
3. Implement interpolation.
4. Analyze the output spectrum.
5. Determine the final sampling rate.
6. Compare input and output signals.

Code of Conduct:

I ____________________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Simulation tools					
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Aim:

To Simulate a multirate system using decimation and interpolation and analyze the resulting sampling rate conversion.

Objective:

To design and simulate a multirate sampling rate conversion system in MATLAB by combining interpolation (L = 2) and decimation (M = 3) on an 18\text{ kHz} input signal. The objective is to implement an optimal anti-imaging/anti-aliasing FIR lowpass filter and analyze the time-domain and spectral characteristics of the converted 12\text{ kHz} output signal.

Program:

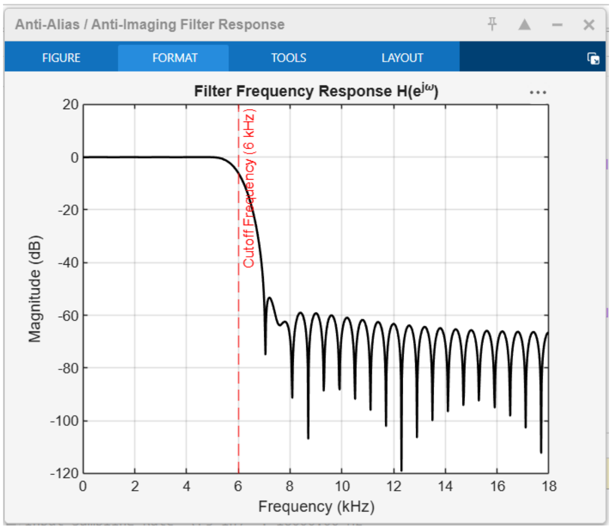
```
% ======================================================================
% Combined Decimation and Interpolation (Rational Rate Conversion)
% Sampling Rate Conversion Factor: L/M = 2/3
% ======================================================================

clear; clc; close all;

%% Given Data
Fs_in = 18000;        % Input Sampling Rate = 18 kHz
L    = 2;             % Interpolation Factor
M    = 3;             % Decimation Factor

%% Task 5: Determine the Final Sampling Rate
% Intermediate rate after upsampling: Fs_inter = L * Fs_in = 36 kHz
Fs_inter = L * Fs_in;
% Final sampling rate after downsampling: Fs_out = Fs_inter / M = 12 kHz
Fs_out   = Fs_in * (L / M);

fprintf('=== SAMPLING RATE ANALYSIS ===\n');
fprintf('Input Sampling Rate  (Fs_in)  : %8.2f Hz\n', Fs_in);
fprintf('Interpolation Factor (L)      : %8d\n', L);
fprintf('Decimation Factor   (M)       : %8d\n', M);
fprintf('Intermediate Rate    (Fs_inter): %8.2f Hz\n', Fs_inter);
fprintf('Final Sampling Rate  (Fs_out) : %8.2f Hz\n\n', Fs_out);

%% Input Signal Generation
% Generate a multi-tone signal containing 800 Hz and 2.5 kHz
% Note: Output Nyquist limit is Fs_out / 2 = 6 kHz. Both tones fit safely.
t_duration = 0.01; % 10 ms duration for time-domain viewing
t_in = 0 : 1/Fs_in : t_duration - 1/Fs_in;
f1 = 800;   % Tone 1: 800 Hz
f2 = 2500;  % Tone 2: 2500 Hz
```

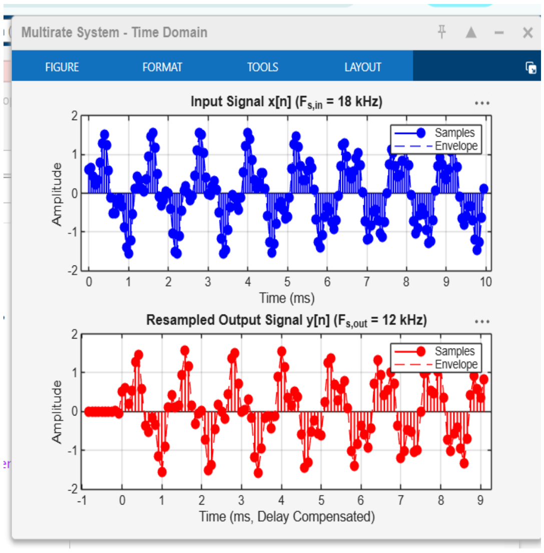
```
x_in = sin(2*pi*f1*t_in) + 0.6*cos(2*pi*f2*t_in);

%% Task 1: Design Anti-Alias / Anti-Imaging Lowpass Filter
% Cutoff frequency must remove images created by upsampling (pi/L)
% and prevent aliasing from downsampling (pi/M).
% Cutoff in normalized units (0 to 1, where 1 is Fs_inter/2):
fc_norm = 1 / max(L, M);  % fc = 1/3 of Fs_inter/2 = 6 kHz
N = 60;                   % FIR Filter order (even for symmetric Type I)

% Linear phase FIR filter designed with Hamming window
% Scale coefficients by L to compensate for zero insertion amplitude drop
b = L * fir1(N, fc_norm);

%% Task 3: Implement Interpolation (Upsampling + Lowpass Filtering)
% Step 3A: Zero Insertion (Upsample by L)
x_up = upsample(x_in, L);

% Step 3B: Anti-Imaging Filtering
x_interp = filter(b, 1, x_up);

%% Task 2: Perform Decimation (Downsampling by M)
% Keep 1 out of every M samples
x_out = downsample(x_interp, M);

% Define time vector for output signal
t_out = 0 : 1/Fs_out : (length(x_out)-1)/Fs_out;

% Account for filter group delay (N/2 samples at Fs_inter)
group_delay_seconds = (N / 2) / Fs_inter;

%% Task 4 & 6: Frequency Spectrum Analysis and Comparison

% FFT parameters
NFFT = 2048;

% Input Spectrum
X_in = fftshift(fft(x_in, NFFT));
f_in_axis = linspace(-Fs_in/2, Fs_in/2 - Fs_in/NFFT, NFFT);

% Intermediate Spectrum (Filtered Upsampled Signal)
X_interp = fftshift(fft(x_interp, NFFT));
f_interp_axis = linspace(-Fs_inter/2, Fs_inter/2 - Fs_inter/NFFT, NFFT);

% Output Spectrum
X_out = fftshift(fft(x_out, NFFT));
f_out_axis = linspace(-Fs_out/2, Fs_out/2 - Fs_out/NFFT, NFFT);
```

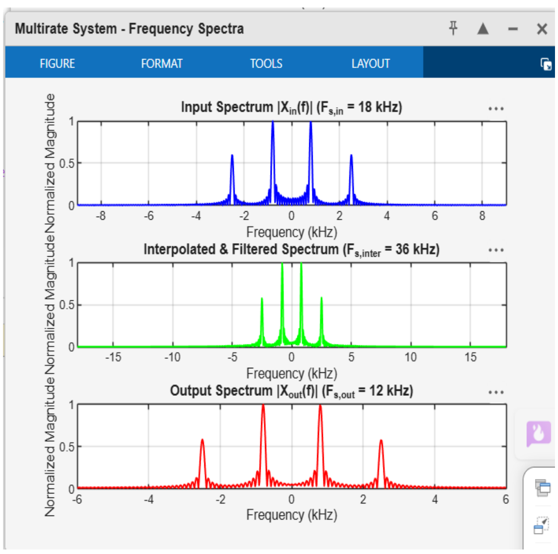
```
%% Plotting Results

% 1. Time-Domain Signal Comparison
figure('Name', 'Multirate System - Time Domain', 'NumberTitle', 'off');

subplot(2,1,1);
stem(t_in*1e3, x_in, 'filled', 'b', 'LineWidth', 1.2);
hold on;
plot(t_in*1e3, x_in, 'b--', 'LineWidth', 0.8);
grid on;
title('Input Signal x[n] (F_{s,in} = 18 kHz)');
xlabel('Time (ms)');
ylabel('Amplitude');
legend('Samples', 'Envelope');

subplot(2,1,2);
% Align time to compensate for FIR filter delay in display
stem((t_out - group_delay_seconds)*1e3, x_out, 'filled', 'r', 'LineWidth', 1.2);
hold on;
plot((t_out - group_delay_seconds)*1e3, x_out, 'r--', 'LineWidth', 0.8);
grid on;
title('Resampled Output Signal y[n] (F_{s,out} = 12 kHz)');
xlabel('Time (ms, Delay Compensated)');
ylabel('Amplitude');
legend('Samples', 'Envelope');

% 2. Spectrum Analysis Comparison
figure('Name', 'Multirate System - Frequency Spectra', 'NumberTitle', 'off');

subplot(3,1,1);
plot(f_in_axis/1e3, abs(X_in)/max(abs(X_in)), 'b', 'LineWidth', 1.2);
grid on;
title('Input Spectrum |X_{in}(f)| (F_{s,in} = 18 kHz)');
xlabel('Frequency (kHz)');
ylabel('Normalized Magnitude');
xlim([-Fs_in/2e3, Fs_in/2e3]);

subplot(3,1,2);
plot(f_interp_axis/1e3, abs(X_interp)/max(abs(X_interp)), 'g', 'LineWidth', 1.2);
grid on;
title(['Interpolated & Filtered Spectrum (F_{s,inter} = ' num2str(Fs_inter/1e3) ' kHz)']);
xlabel('Frequency (kHz)');
ylabel('Normalized Magnitude');
xlim([-Fs_inter/2e3, Fs_inter/2e3]);

subplot(3,1,3);
```

```
plot(f_out_axis/1e3, abs(X_out)/max(abs(X_out)), 'r', 'LineWidth', 1.2);

grid on;

title('Output Spectrum |X_{out}(f)| (F_{s,out} = 12 kHz)');

xlabel('Frequency (kHz)');

ylabel('Normalized Magnitude');

xlim([-Fs_out/2e3, Fs_out/2e3]);


% 3. Filter Magnitude Response

figure('Name', 'Anti-Alias / Anti-Imaging Filter Response', 'NumberTitle', 'off');

[H, freq] = freqz(b, 1, 1024, Fs_inter);

plot(freq/1e3, 20*log10(abs(H)/L), 'k', 'LineWidth', 1.5);

grid on;

title('Filter Frequency Response H(e^{j\omega})');

xlabel('Frequency (kHz)');

ylabel('Magnitude (dB)');

xline((Fs_inter/2 * fc_norm)/1e3, 'r--', 'Cutoff Frequency (6 kHz)');
```

Result







Conclusion

Filter Optimization: A single 60^{th}-order linear-phase FIR lowpass filter with a cutoff frequency of $f_c = 6\text{ kHz}$ ($\omega_c = \frac{\pi}{3}$) placed between the upsampler and downsampler effectively served a dual purpose: suppressing image frequencies created during upsampling (L=2) and preventing aliasing prior to downsampling (M=3).